Introduction to Robotics: Perception Part 2 – Modeling of Sensors and Actuators

Shuai Zhang

Department of Computer Science
University of Central Oklahoma

Lecture Plan

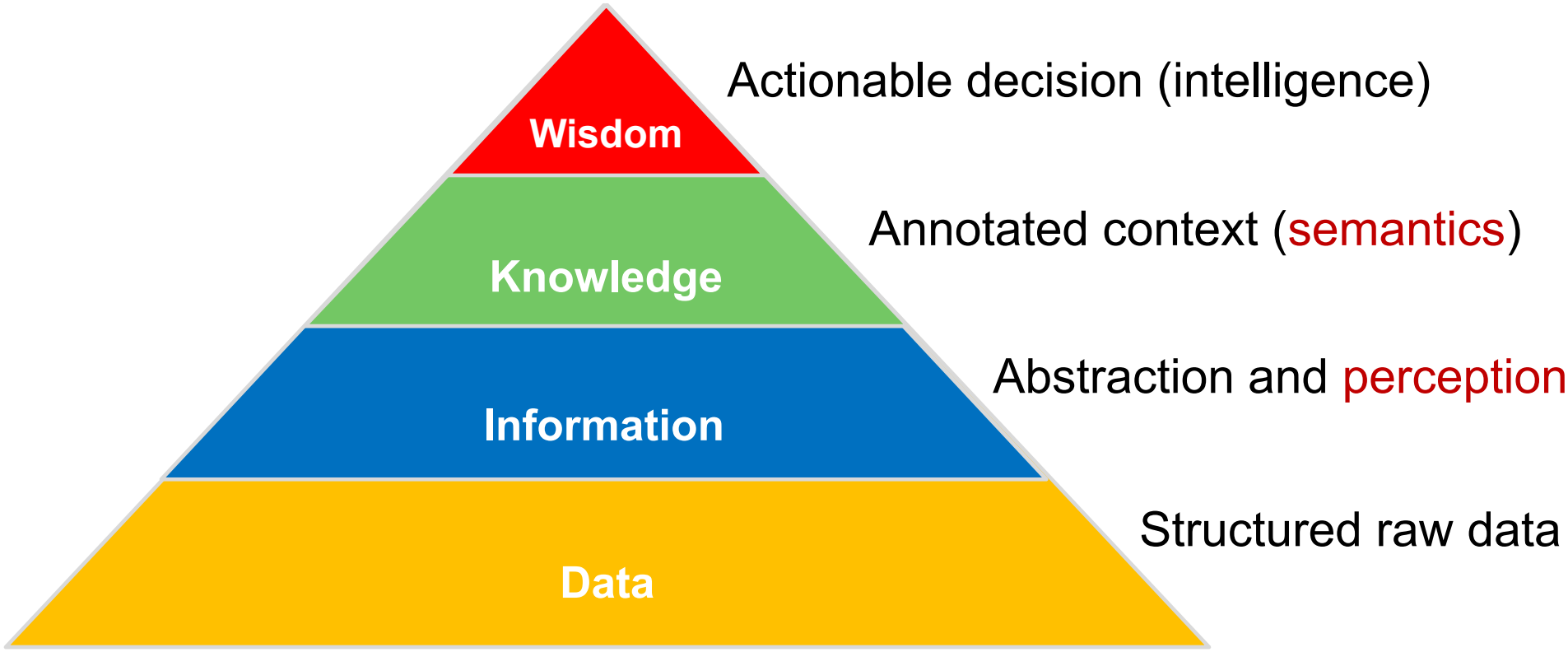
- Examples of Sensors
- Examples of Actuators
- Errors in Sensors and Actuators
- Make Use of Sensors/Actuators
- Modeling of Sensors and Actuators
- Sensor and Actuator Classes in EV3

Modeling of Sensors and Actuators 1

- Modeling is an important task in Software Engineering
  - Object Oriented Concepts/Programming
    - Good modularity and reusability
  - Unified Modeling Language (UML)
    - Articulate complex ideas succinctly and precisely
  - Involve procedures like
    - Requirement gathering / analysis
    - Design
    - Implementation
    - Testing
    - Management
    - …

Modeling of Sensors and Actuators 2

- Look at our tasks again: what are involved?



The cognitive pyramid

Perceptions

- The perception model takes as input raw data earned from sensors
  - Generic scene segmentation
    - E.g. Road branch around an intersection
  - Instance localization/tracking
    - E.g. lane detection, car tracking
  - Concept detection
    - Example of concepts: events, physical objects, etc.

Breux, Yohan, Sébastien Druon, and Rene Zapata. "From Perception to Semantics: An Environment Representation Model Based on Human-Robot Interactions." *2018 27th IEEE International Symposium on Robot and Human Interactive Communication (RO-MAN)*. IEEE, 2018.

Semantics 1

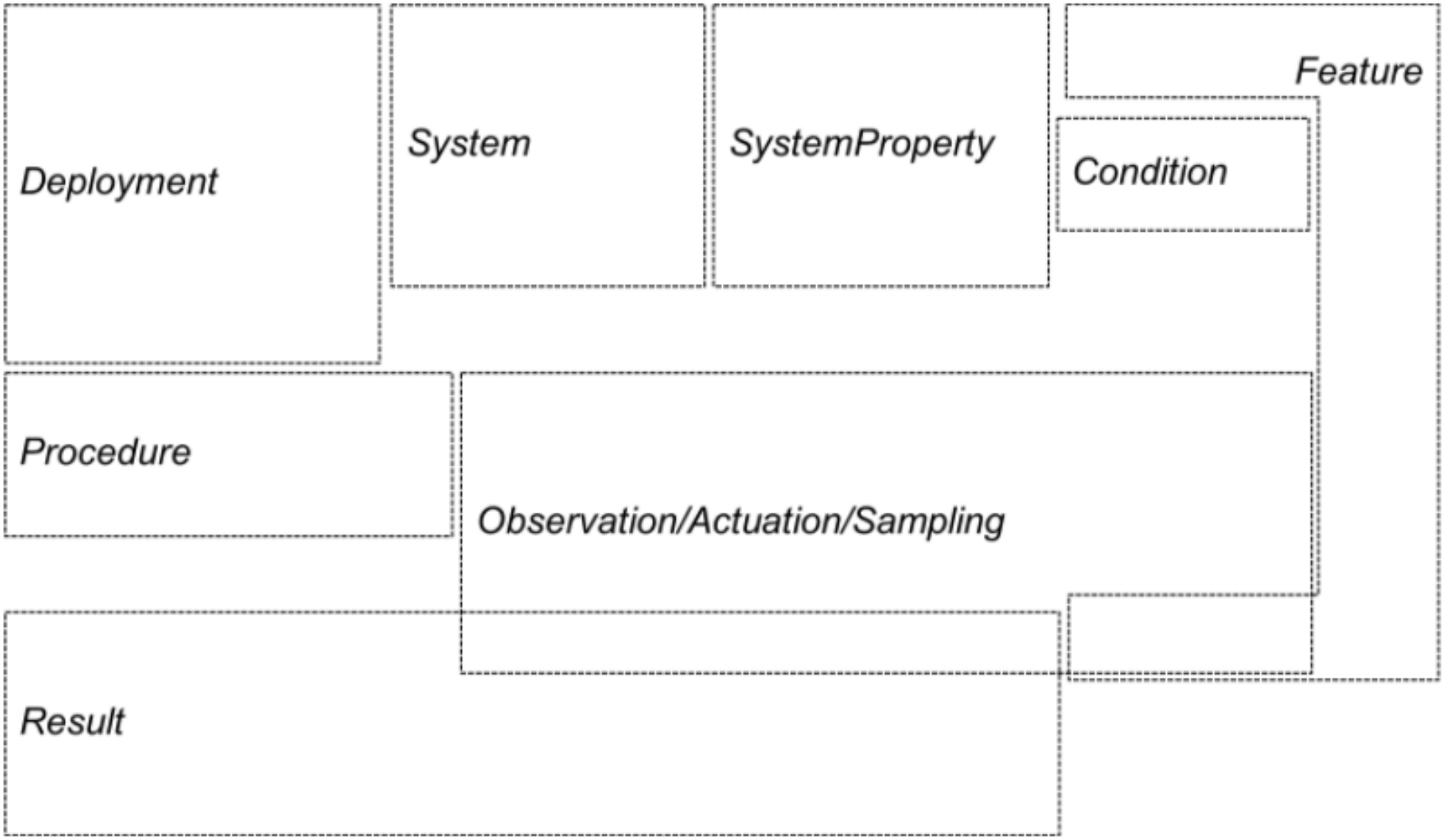
- Descriptive information about concepts and relations between concepts and sub-concepts
  - Typically in form of an ontology
    - Ontology is the philosophical study of being.
  - We need to have a clear specification of the concepts
    - It has the similar idea as the software requirements specification in software engineering: a description of a software system to be developed
    - The specification can be a shared understanding about concepts among different parties involved in a specific task -> the domain ontology could be a template
    - Includes both outline and details
      - E.g. components, functionalities
      - Ideally it can be parsed automatically

Support

Semantics 2

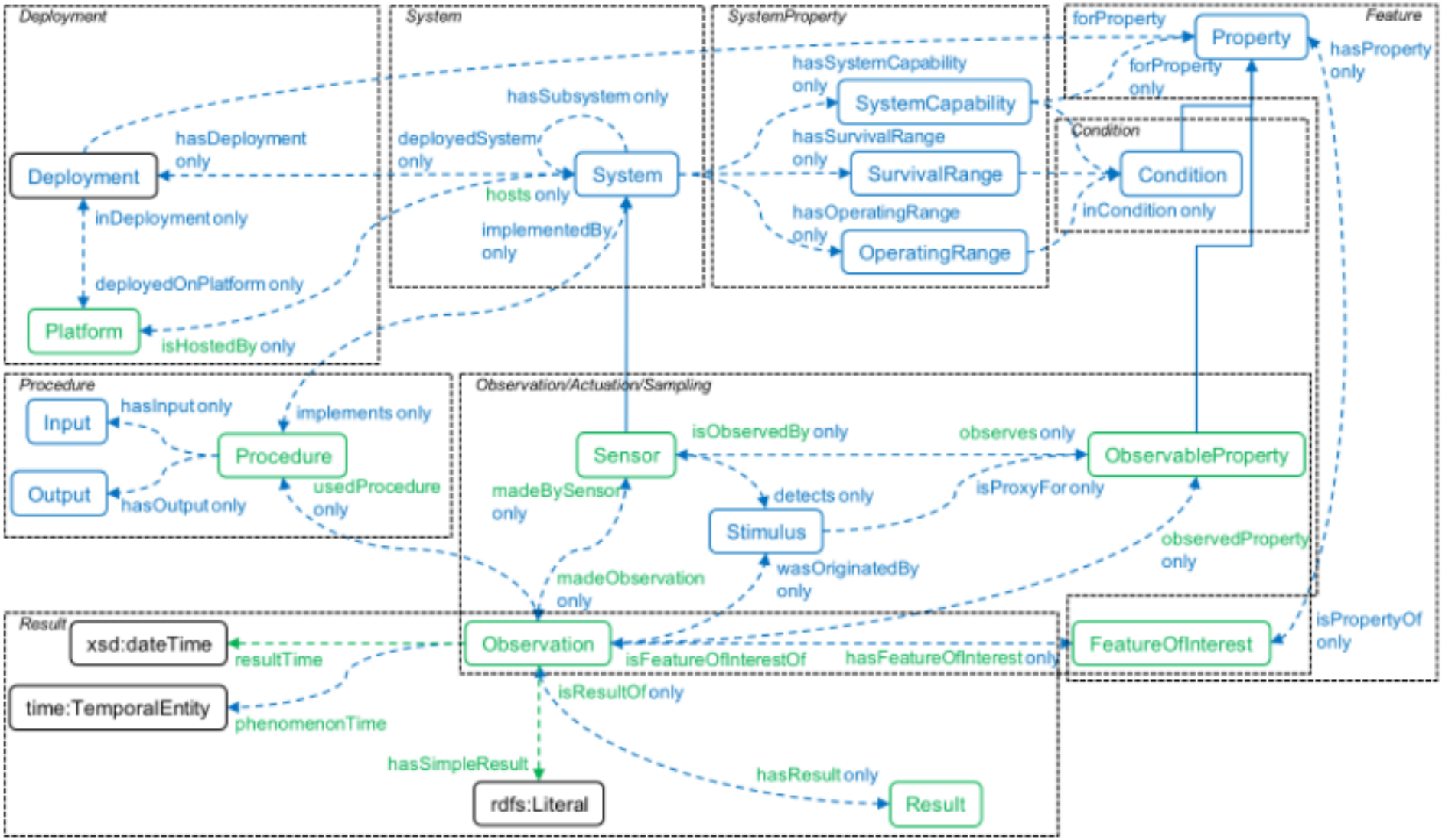
- The most widely used semantic model for sensor and actuator is Semantic Sensor Network (SSN) Ontology
  - https://www.w3.org/TR/vocab-ssn/
  - Expressive representation of sensor observations, sampling, actuation, etc.
    - Sensor devices
    - Systems
    - Processes
    - Observations
    - …
  - Light weighted
  - Useful for heterogeneous environment, e.g. IoT

Semantics 3



Overview of SSN modules

Semantics 4



Overview of SSN classes and properties (sensor perspective)

Semantics 5

- How to model the errors of a system component?
  - System can ssn:hasSubsystem: subsystem is also an instance of system
  - A System instance can ssn-system:hasSystemProperty
    - hasSystemProperty is a subclass of ssn:hasProperty relation
  - ssn-system:inCondition relation can be used to specify the ssn-system:accuracy, which is a subclass of ssn-system:condition
    - Regarding the accuracy itself: it can be sosa:FeatureOfInterest
  - ssn-system:condition is a subclass of ssn:Property
  - All these work together, and they can model the errors of a system

Semantics 6

- How about accuracy as a ssn:Property?

<DHT22/4578#TemperatureSensorAccuracy> a ssn:Property , ssn-system:Accuracy , schema:PropertyValue ;

rdfs:comment "The accuracy of the temperature sensor is +-0.5 ° C in normal temperature and humidity conditions."@en ;

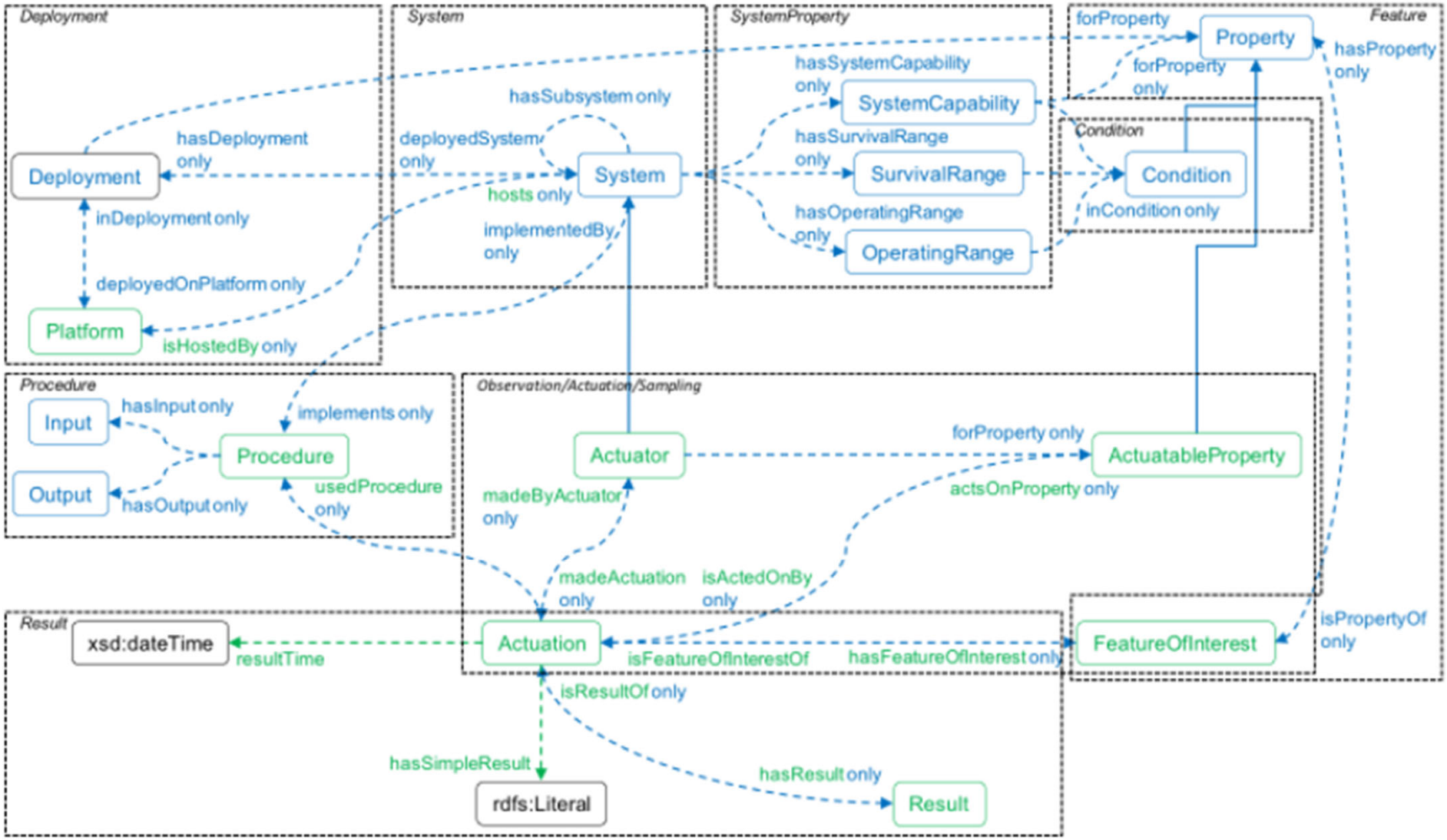
schema:minValue -0.5 ;

schema:maxValue 0.5 ;

schema:unitCode qudt-unit-1-1:DegreeCelsius .

- Why ssn-system:condition is a subclass of ssn:Property ?
  - Think about name or ID of a system component
    - No change through time
- The spirit of modeling

Semantics 7



Overview of SSN classes and properties (actuation perspective)

Semantics 8

- Example of using SSN: iPhone Barometer
  - https://www.w3.org/TR/vocab-ssn/integrated/examples/iphone_barometer-sosa.ttl

```
# An iPhone 7 as the Platform that hosts several sensors,
# among others the Bosch Sensortec BMP282 atmospheric pressure sensor.

<iphone7/35-207306-844818-0> a sosa:Platform ;
  rdfs:label "IPhone 7 - IMEI 35-207306-844818-0"@en ;
  rdfs:comment "IPhone 7 - IMEI 35-207306-844818-0 - John Doe"@en ;
  sosa:hosts <sensor/35-207306-844818-0/BMP282> .

<sensor/35-207306-844818-0/BMP282> rdf:type sosa:Sensor ;
  rdfs:label "Bosch Sensortec BMP282"@en ;
  sosa:observes <sensor/35-207306-844818-0/BMP282/atmosphericPressure>
.
```

Semantics 9

- Example of using SSN: iPhone Barometer (cont'd)

```
<Observation/346345> rdf:type sosa:Observation ;
  sosa:observedProperty <sensor/35-207306-844818-0/BMP282/atmosphericPressure> ;
  sosa:hasFeatureOfInterest  <earthAtmosphere> ;
  sosa:madeBySensor <sensor/35-207306-844818-0/BMP282> ;
  sosa:hasResult [
    rdf:type qudt-1-1:QuantityValue ;
    qudt-1-1:numericValue "101936"^^xsd:double ;
    qudt-1-1:unit qudt-unit-1-1:Pascal ] ;
  sosa:resultTime [
    rdf:type time:Instant ;
    time:inXSDDateTimeStamp "2017-06-06T12:36:13+00:00“ ^^xsd:dateTimeStamp ] .
```

Knowledge representation: readability is not good just like assembly

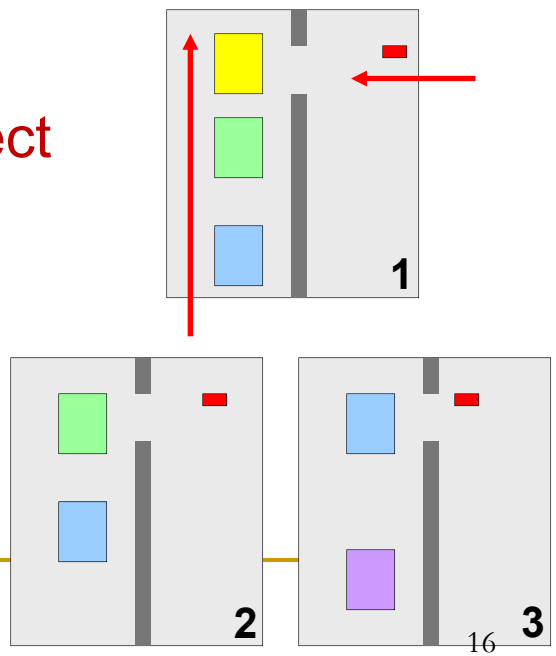
Semantics 10

- Actuation example: apartment with automatic window
  - https://www.w3.org/TR/vocab-ssn/integrated/examples/apartment-134.ttl

- Some analysis
  - Hard to read for human (needs better languages)
  - Easy to read by program
    - Standard format (OWL, RDF, etc.): edit in **Protege**
    - Standard annotations (aligned with W3C standard)
    - Work with prevailing reasoning languages and knowledge bases, e.g. **Jena**, **Prolog**, etc.
      - Enable automatic deployment and management
      - Build context information for high-level applications

Semantics 11

- An example of high-level application: Behavior Learning and Prediction
  - Forward view
    - Objects in front and further
    - Each vehicle perform their own analysis and aggregate the derived object information to obtain global information
  - Side view
    - Multiple vehicles identify the same object by semantic context information
    - Fuse the motion information of each object from multiple vehicles to obtain the speed and trajectory



Fuses data from ⇒ Speed of

⇒ Potential collision

Sensor and Actuator Classes in EV3 1

- Installation of leJOS (Lego Mindstorms Java Operating System)
  - On Windows: https://sourceforge.net/p/lejos/wiki/Windows%20Installation/
    - The current version of leJOS is 0.9.1
    - Java 7 is recommended for this release
    - The SD card is ready
  - On Linux or Mac OS: https://sourceforge.net/projects/lejos/files/lejos-EV3/0.9.0-beta/
    - The current version of leJOS is 0.9.0

Sensor and Actuator Classes in EV3 2

- Sensor Framework
  - Mode
    - The type of measurement a sensor can make
    - A sensor can have one or more modes
      - E.g. The color sensor has four modes: Color ID, Red, RGB, Ambient https://web.suffieldacademy.org/cs/lejos_ev3_api/lejos/hardware/sensor/EV3ColorSensor.html
  - Sample
    - A single measurement is called a sample
    - A sample consists of one ore more values taken at one moment
      - The color sensor in RGB mode, can read 3 values at a time
    - SampleProvider
      - Fetch and process multiple samples: bulk read

Sensor and Actuator Classes in EV3 3

- Sensor Framework (cont'd)
  - Only modes can take measurements, the sensor by itself does not.
  - A combination of a sensor and a mode will define a series of measurements
  - How to define the class hierarchy?
    - SampleProvider: interface

```
|_____ SensorMode: interface
        |_______ SensorModes: interface
                 |_______ BaseSensor: class
                          __________|__________
                          |         |         |
                 AnalogSensor   ….   UARTSensor
```

Sensor and Actuator Classes in EV3 4

- Actuator
  - Motor
    - Built-in encoders: how far a motor has rotated
    - Motors with **encoders**: **regulated** motors
      - Unregulated motor: just set power
        - But two motors perform differently at the same power
    - Encoder: interface
      |_____ Tachometer: interface
      |_____ RegulatedMotor: interface
      |_____ BaseRegulatedMotor: class

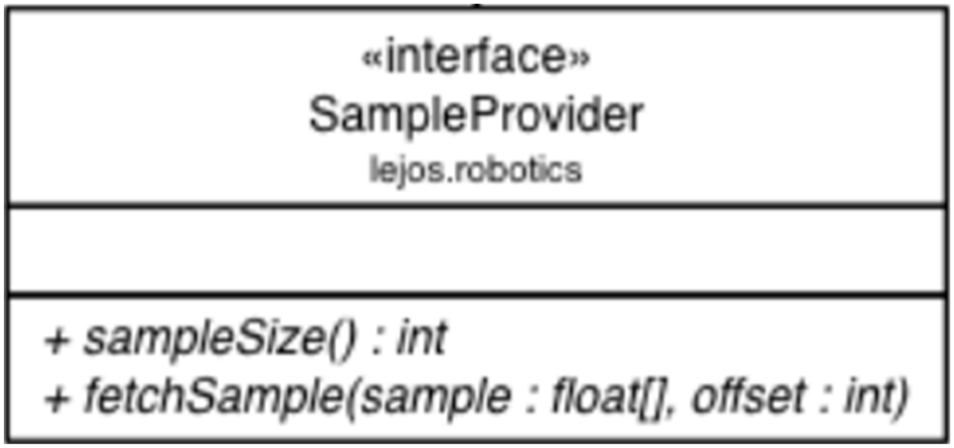
Sensor Classes 1

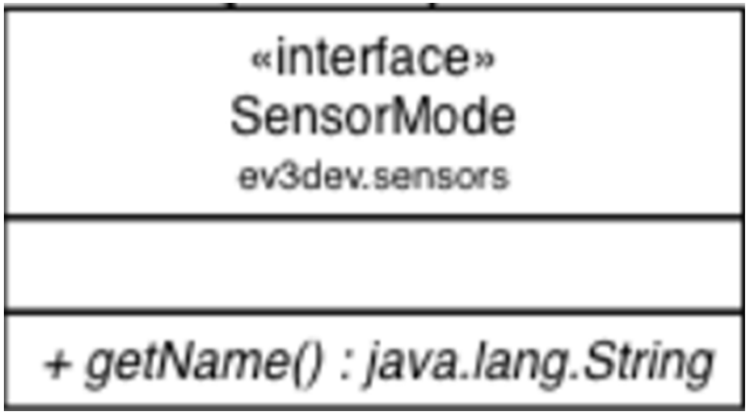
BaseSensor
ev3dev.sensors

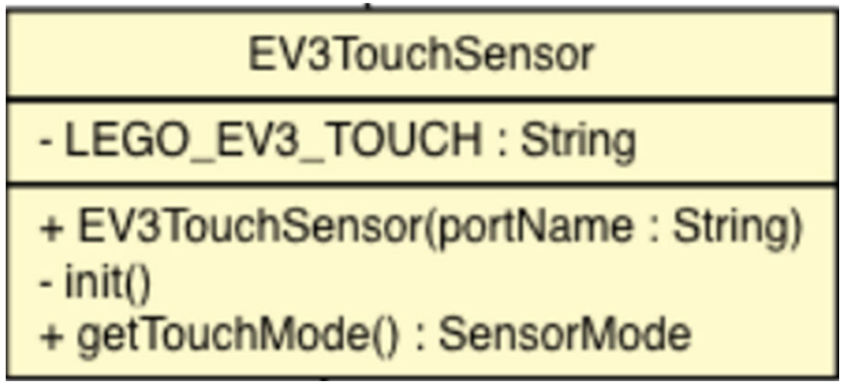
```
# SWITCH_DELAY : int
# currentMode : int
# currentModeS : java.lang.String
# modes : ev3dev.sensors.SensorMode[]
~ modeList : java.util.ArrayList<java.lang.String>
```

```
+ BaseSensor(sensorPort : Port, mode : java.lang.String, device : java.lang.String)
+ BaseSensor(sensorPort : Port, mode : java.lang.String)
# setModes(m : ev3dev.sensors.SensorMode[])
+ getAvailableModes() : java.util.ArrayList<java.lang.String>
+ getMode(mode : int) : ev3dev.sensors.SensorMode
+ getMode(modeName : java.lang.String) : ev3dev.sensors.SensorMode
- isValid(mode : int) : boolean
- getIndex(modeName : java.lang.String) : int
+ getName() : java.lang.String
+ sampleSize() : int
+ fetchSample(sample : float[], offset : int)
+ setCurrentMode(mode : int)
+ setCurrentMode(modeName : java.lang.String)
+ getCurrentMode() : int
+ getModeCount() : int
# switchMode(newMode : int, switchDelay : long)
# switchMode(newMode : java.lang.String, switchDelay : long)
```

Sensor Classes 2








```
touch = new EV3TouchSensor(SensorPort.S1)
SampleProvider sp = touch.getTouchMode()
float[] sample = new float[sp.sampleSize()]
sp.fetchSample(sample, 0)
```

Sensor Classes 3

EV3IRSensor

- LEGO_EV3_IR : String

+ EV3IRSensor(portName : Port)
- init()
+ getDistanceMode() : SensorMode
+ getSeekMode() : SensorMode
+ getRemoteMode() : SensorMode

EV3UltrasonicSensor

- DISABLED : int
- LEGO_EV3_US : String

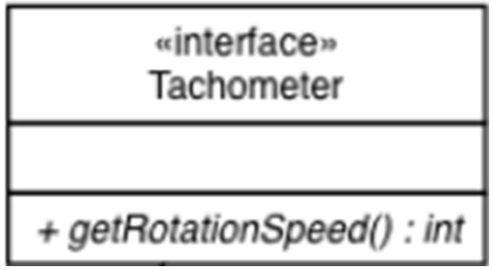
+ EV3UltrasonicSensor(portName : String)
- init()
+ getListenMode() : SampleProvider
+ getDistanceMode() : SampleProvider
+ enable()
+ disable()
+ isEnabled() : boolean

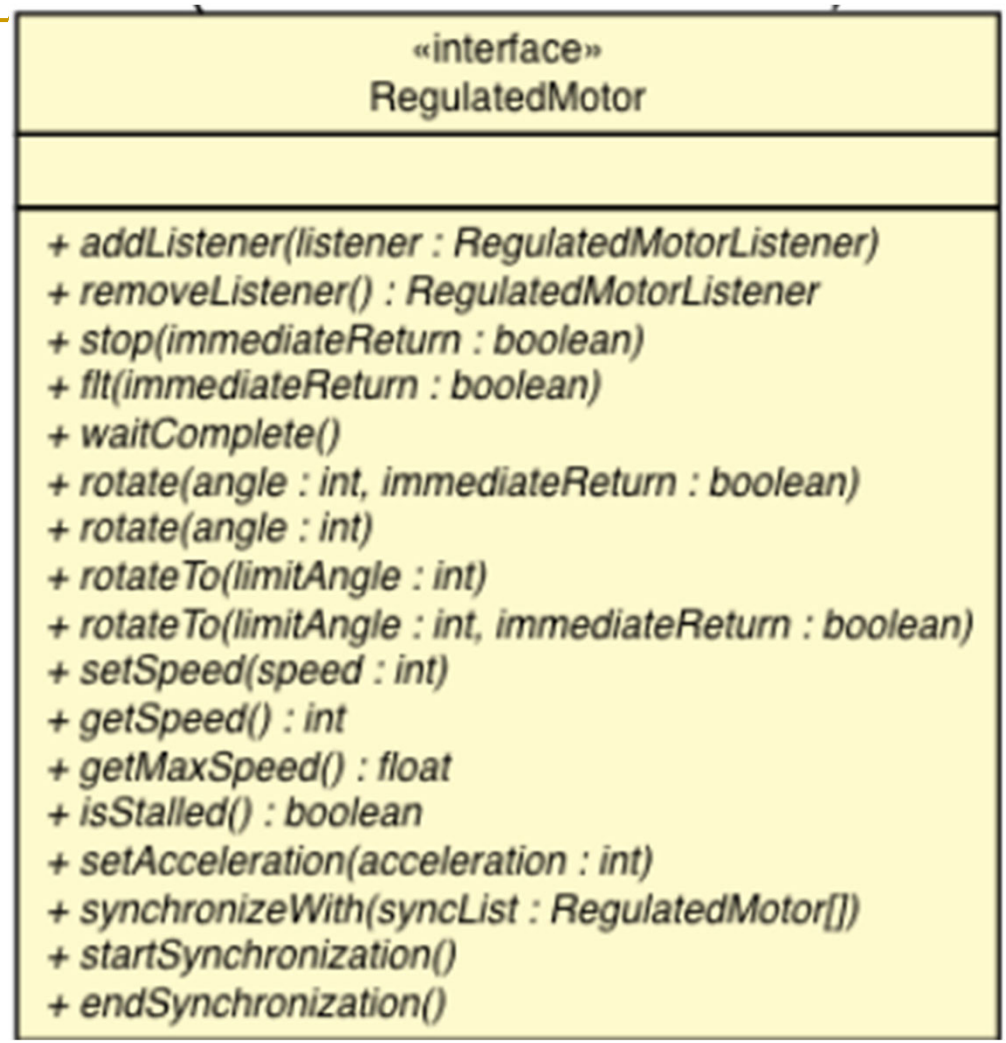
EV3ColorSensor

colorMap : int[]
- LEGO_EV3_COLOR_SENSOR : String
- COL_RESET : String
- COL_REFLECT : String
- COL_AMBIENT : String
- COL_COLOR : String
COL_REFRAW : String
- COL_RGBRAW : String
COL_CAL : String
- lightColor : int[]
raw : short[]

+ EV3ColorSensor(portName : Port)
- initModes()
+ getColorID() : int
+ setFloodlight(floodlight : boolean)
+ isFloodlightOn() : boolean
+ getFloodlight() : int
+ setFloodlight(color : int) : boolean
+ getColorIDMode() : SensorMode
+ getRedMode() : SensorMode
+ getAmbientMode() : SensorMode
+ getRGBMode() : SensorMode

Motor Classes






```
mA = new EV3LargeRegulatedMotor(MotorPort.A)
mA.setSpeed(500);
mA.brake();
mA.forward();
```